Slide 1

Defining Big Data

Extremely large datasets that are complex and difficult to manage using traditional tools

5 V's of Big Data:

Volume

Velocity

Variety

Veracity

Value

Difference between Data, Statistics, and Analytics

Data: Raw observations (e.g., sales numbers) v Statistics: Summarizing and describing data (e.g., average sales) v Analytics: Drawing insights, making predictions (e.g., future demand forecasting

Data Types

Structured Data: Organized into rows/columns (e.g., databases)

Unstructured Data: Free-form text, images, audio

Semi-structured Data: Has some organizational properties (e.g., JSON, XML)

Metadata: Data about data

ASCII and Unicode

ASCII: Uses 7-bit encoding, supports 128 characters

Suitable for English text

Unicode: Universal character encoding standard

Supports characters from most world languages

Variants: UTF-8, UTF-16

Slide 2

DIKW Pyramid

Data: Raw symbols or facts (e.g., numbers, words)

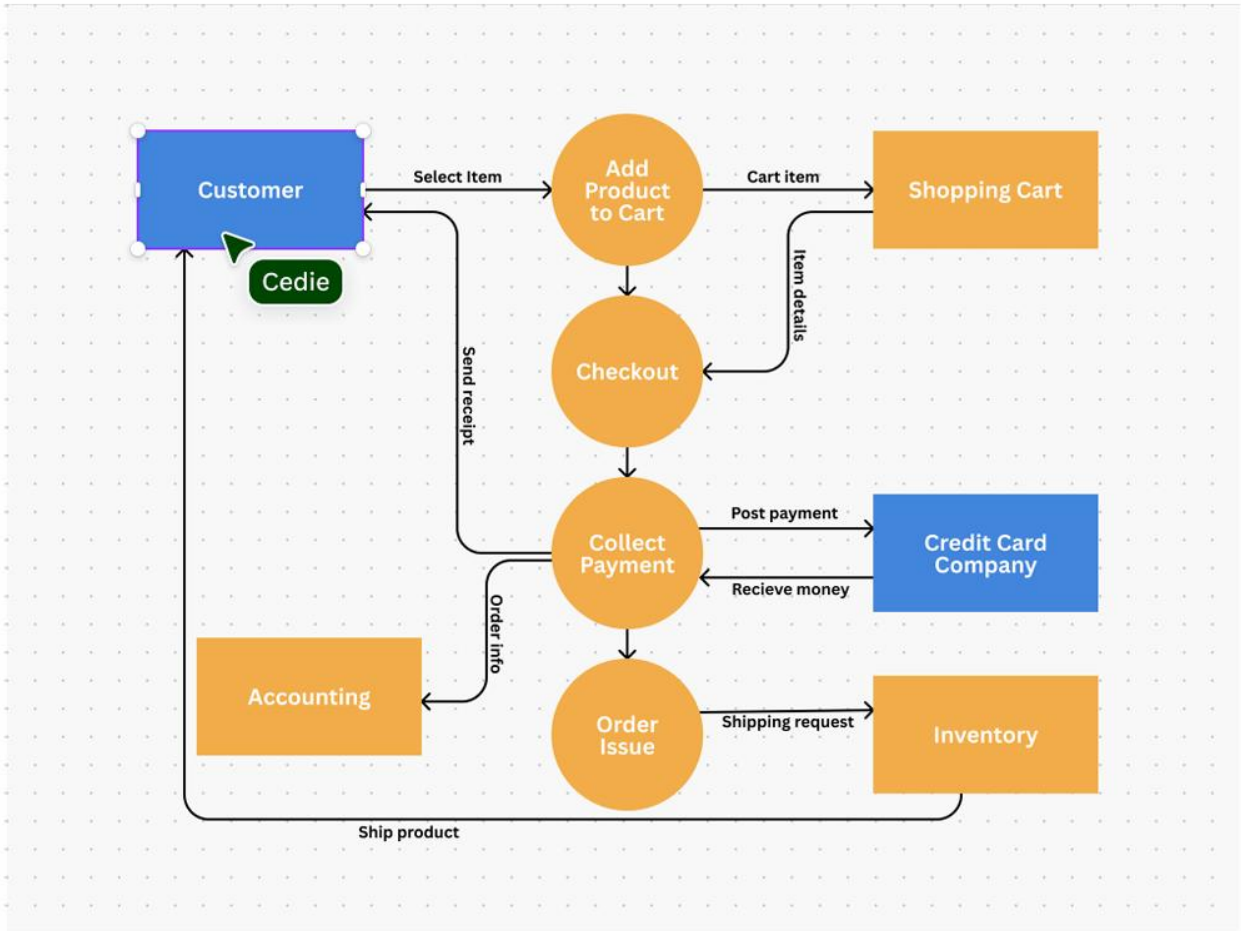
Information: Data given meaning through context

Knowledge: Application of information to answer "how"

Wisdom: Understanding of “why” and making informed decisions

Data Flows and Data Diagrams

Symbol	Name	Description
🟦 Rectangle	**External Entity**	An outside source or destination of data (e.g., user, system, organization)
🔵 Circle / Rounded Rectangle	**Process**	A major process or function that transforms input into output
🟨 Open-ended Rectangle	**Data Store**	A place where data is stored (e.g., database, file)
➡️ Arrow	**Data Flow**	Movement of data between entities, processes, or data stores



Slide 3

What Are Linear Data Structures?

Linear data structures are types of data arrangements in which elements are organized sequentially — one after another — so that each element is connected to its previous and next element (except the first and last ones).

In this structure, data items are stored in a linear order, and they can be traversed sequentially, making the access pattern predictable and straightforward.

Types of linear data structures:

- Arrays
- Linked Lists
- Stacks
- Queues

Definition: An array is a collection of elements stored in contiguous memory locations, where each element can be directly accessed using an index.

- Advantages:
    - Simple and easy to use.
    - Fast access to elements using the index (constant time, O(1)).
    - Useful for static data storage where the size is known beforehand.
    - Efficient for reading and iterating through elements.
- Disadvantages:
    - Fixed size — cannot grow or shrink dynamically.
    - Insertion and deletion are costly (require shifting elements).
    - Memory wastage if array size is overestimated.

Inefficient for searching for unsorted data (O(n) time).

Linked List

Definition: A linked list is a collection of nodes, where each node contains data and a reference (pointer) to the next node. (Types: Singly Linked List, Doubly Linked List, Circular Linked List).

Advantages:

- Dynamic in size — can easily grow or shrink at runtime.
- Efficient insertion and deletion (no shifting required).
- Memory utilization is optimized; no pre-allocation required.

Disadvantages:

- Sequential access only (no direct indexing).
- Requires extra memory for storing pointers.
- More complex to implement and manage.
- Reverse traversal is difficult in singly linked lists.

Stack

Definition: A stack is a LIFO (Last In, First Out) structure where insertion and deletion occur only at one end called the “top.”

Advantages:

- Simple and easy to implement using arrays or linked lists.
- Provides control over data access (only top element).
- Useful in recursion, expression evaluation, and backtracking.
- Memory efficient — follows a predictable access pattern.

Disadvantages:

- Limited access (only top element can be accessed).
- Fixed size if implemented using arrays.
- Overflow and underflow conditions may occur.

Queue

Definition: A queue is a FIFO (First In, First Out) structure where insertion occurs at the rear and deletion at the front.

Advantages:

- Maintains order of processing (first come, first served).
- Ideal for scheduling and resource allocation.
- Prevents data loss by maintaining a waiting line mechanism.

Disadvantages:

- Random access not possible.
- Fixed size if implemented using arrays.
- May become full (overflow) even if space is available (in simple queue).

What Are Tree Structures?

A Tree Structure is a non-linear data structure that organizes data in a hierarchical form, like an inverted tree in nature — starting from a single root node and expanding into branches (edges) that connect to child nodes.

Basic Concepts of Tree Structures

- Root Node: The topmost node in the tree (has no parent).
- Child Node:A node that descends from another node.
- Parent Node: A node that has one or more child nodes.
- Leaf Node: A node with no children.
- Edge: The link between a parent and a child node.
- Level: The depth of a node in the tree (root is at level 0).
- Subtree: Any node and all its descendants form a subtree.

Types of Tree Structures

- General Tree: Each node can have any number of children.
- Binary Tree: Each node has at most two children (left and right).
- Binary Search Tree (BST): A binary tree where the left child < parent < right child.

- Balanced Tree (e.g., AVL, Red-Black Tree): A binary tree that maintains balanced height for efficiency.
- Heap: A complete binary tree used for priority operations.
- Trie (Prefix Tree):Used for storing strings efficiently (like in autocomplete).

What Are Index and Pointer Structures?

Index and Pointer structures are two important data organization techniques used to manage data efficiently in memory or storage.
They help in fast data access, retrieval, and linking between data elements.

- Used in:
    - File systems
    - Databases
    - Graphs
    - Memory allocation

Index Structure

An Index is a data structure that improves the speed of data retrieval operations on a database, array, or file by maintaining a separate list (index) of key values and their corresponding memory or record locations.

Think of an index like the index of a book — it lists keywords (topics) along with the page numbers where you can find them.

Common Index Structures

- B-Tree / B+ Tree Index – used in databases for ordered data retrieval.
- Hash Index – uses hash functions for fast equality searches.
- Inverted Index – used in search engines for keyword-based retrieval.

Advantages

- Fast search, retrieval, and sorting.
- Reduces disk access in large databases.
- Efficient range queries (e.g., finding values between 100–200).

Disadvantages

- Requires extra memory or disk space.
- Needs updating whenever data changes.
- Overhead during insertion and deletion.

Pointer Structure

A Pointer is a variable that stores the address (location) of another variable, record, or node in memory.

In data structures, pointers are used to link elements dynamically, enabling non-contiguous memory storage.

Imagine a chain — each link (node) knows where the next one is located through a pointer.

Common Uses of Pointers

- Linked Lists (Singly, Doubly, Circular)
- Trees (pointers to left and right child)
- Graphs (adjacency representation)
- Dynamic Memory Allocation (malloc, new)

Advantages

- Efficient dynamic memory use.
- Enables creation of flexible and complex structures (lists, trees, graphs).
- Supports non-contiguous data storage.

Disadvantages

- Increases program complexity.
- Pointer errors (dangling or null pointers) can cause crashes.
- Harder to debug.

Slide 4

Population

Population refers to the entire set of data points or observations that a data scientist aims to analyze or draw conclusions about.
It represents the complete group that possesses one or more characteristics of interest.
It can be finite or infinite.

- Example:
    - All customer transactions of an e-commerce company in 2025.
    - All students in a university
    - All users of an app

All possible outcomes of a dice roll

Type	Description	Example
Random Sampling	Every element has equal chance of being selected	1,000 random Netflix users
Stratified Sampling	Population divided into groups (strata) and samples taken from each	Users grouped by region or gender
Systematic Sampling	Every kth element from ordered list	Every 10th transaction
Cluster Sampling	Divide population into clusters and randomly select entire clusters	Select few universities for student survey
Convenience Sampling	Based on ease of access (biased)	Surveying people at your own university gate

Sample

- A sample is a subset of the population selected for analysis and modeling.
- It represents the population when it's impractical or expensive to collect all data.
- Sampling allows data scientists to make inferences, train models, and validate hypotheses efficiently.
- Example: Selecting 5,000 customer reviews from 5 million total reviews on Amazon.

What is Statistical Modeling

Statistical Modeling is the process of using mathematical equations to represent real-world data relationships.

It helps data scientists to understand patterns, make predictions, and draw conclusions from data.

A model expresses how a dependent variable (output) changes with one or more independent variables (inputs).

- Examples:
    - Regression models (e.g., price prediction)
    - Probability distributions (e.g., binomial, normal)